

WEB APPLICATION DEVELOPMENT - II

Question Bank with Answers

Malla Reddy Technical Campus

Subject	Web Application Development - II (WAD-II)
Modules	I to V
Short Answer	2 Marks — Brief and to the point
Long Answer	5 Marks — Explained with examples

MODULE – I

JavaScript Core Concepts

SHORT ANSWER QUESTIONS (2 Marks Each)

Q1. Explain briefly about Hoisting in JavaScript with an example. [2 Marks]

Ans:

Hoisting is JavaScript's default behavior of moving variable and function **declarations** to the top of their scope before execution. Only declarations are hoisted, not initializations.

Example: `console.log(a); // undefined (not error)`

```
var a = 10;
console.log(a); // 10
```

With **var** — declaration is hoisted but value is undefined. With **let/const** — hoisted but not initialized (Temporal Dead Zone). Functions are fully hoisted.

Q2. What is a Variable? Explain its scope with an example. [2 Marks]

Ans:

A **variable** is a named container that stores data. JavaScript has three types of scope:

- 1. Global Scope:** Declared outside functions — accessible everywhere.
- 2. Function Scope:** Declared inside a function using **var** — accessible only inside that function.
- 3. Block Scope:** Declared with **let/const** inside `{ }` — accessible only in that block.

Example:

```
let globalVar = 'global'; // Global
function test() { var funcVar = 'local'; } // Function scope
if(true) { let blockVar = 'block'; } // Block scope
```

Q3. What problem does asynchronous JavaScript solve in single-threaded execution? [2 Marks]

Ans:

JavaScript is **single-threaded** — it can only execute one task at a time. Without **async**, long tasks (like API calls) would **block** the entire program.

Asynchronous JavaScript solves this by allowing tasks like network requests, timers, and file reads to run in the background. The main thread continues executing other code, and once the **async** task finishes, its callback/result is handled via the **Event Loop**. This keeps the UI responsive.

Q4. How does the 'this' keyword work in JavaScript? Explain with an example. [2 Marks]

Ans:

The **this** keyword refers to the object that is currently calling the function. Its value depends on how the function is called.

Example:

```
const person = {
  name: 'Alice',
  greet() { console.log('Hello, ' + this.name); }
};
person.greet(); // Output: Hello, Alice
```

Here, **this** refers to the **person** object because the method is called on it.

Q5. Explain the types of Destructuring in JavaScript with examples. [2 Marks]

Ans:

Destructuring is a way to **extract values from arrays or objects and assign them to variables easily**.

1. Array Destructuring: Uses [] — unpacks by position.

```
const [a, b] = [10, 20]; // a=10, b=20
```

2. Object Destructuring: Uses { } — unpacks by property name.

```
const {name, age} = {name:'Bob', age:25}; // name='Bob', age=25
```

3. Function Parameter Destructuring: Destructure directly in function params.

```
function greet({name}) { console.log(name); }
```

Q6. Explain Object Destructuring in JavaScript with an example. [2 Marks]

Ans:

Object Destructuring extracts properties from an object into distinct variables using curly braces {}.

Syntax: const { prop1, prop2 } = objectName;

Example:

```
let student = { name: 'Riya', age: 20 };
```

```
let { name, age } = student;
```

```
console.log(name); // Riya
```

```
console.log(age); // 20
```

You can also rename: **let { name: studentName } = student;**

Q7. What is the syntax for Arrow Function in JavaScript? Explain with an example. [2 Marks]

Ans:

Arrow functions (introduced in ES6) provide a shorter syntax for writing functions. They use the => symbol.

Syntax: const funcName = (params) => expression;

Example:

```
const multiply = (a, b) => a * b;
```

```
console.log(multiply(4, 5)); // Output: 20
```

If only one parameter: **const square = x => x * x;**

If multiple lines: use curly braces and **return** keyword.

LONG ANSWER QUESTIONS (5 Marks Each)

Q1. Explain Default Binding and Implicit Binding of 'this' keyword with examples. [5 Marks]

Ans:

1. Default Binding:

Default binding occurs when a regular function is called **without any object reference**

Example:

```
function showThis() {
```

```
  console.log(this); // Window object
```

```
}
```

```
showThis();
```

2. Implicit Binding:

Implicit binding occurs when a function is called **as a method of an object**.

Example:

```
const person = {
```

```
  name: 'Alice',
```

```
  showName() {
```

```
    console.log('Hello, my name is ' + this.name);
```

```
}  
};  
person.showName(); // Output: Hello, my name is Alice
```

Key Rule: Implicit binding is lost when a method is assigned to a variable and called without the object reference — in that case, default binding applies.

Example of binding loss:

```
const fn = person.showName;  
fn(); // Output: Hello, my name is undefined (this = Window)
```

Q2. How is Error Handling done in JavaScript? Explain the error handling mechanisms. [5 Marks]

Ans:

Errors in JavaScript occur due to syntax mistakes, wrong logic, or undefined variables. JavaScript uses the **try...catch...finally** statement to handle errors gracefully.

Types of Errors:

- **SyntaxError:** Wrong code grammar
- **ReferenceError:** Variable not defined
- **TypeError:** Wrong value type used

Error Handling in JavaScript is the process of detecting, managing, and responding to runtime errors so that the program does not crash unexpectedly.

Syntax:

```
try {  
  // risky code  
} catch(error) {  
  // handle error  
} finally {  
  // always runs  
}
```

try block: Contains code that might throw an error.

catch block: Runs only if an error occurs in the try block. Receives the error object with message details.

finally block: Runs always — whether error occurred or not. Used for cleanup (closing connections, stopping timers).

Example:

```
try {  
  console.log(a); // 'a' is not defined  
} catch(error) {  
  console.log('Error: ' + error.message);  
} finally {  
  console.log('Done');  
}  
// Output: Error: a is not defined  
// Done
```

Custom Errors: You can also throw your own errors using **throw new Error('message');**

Q3. Explain the Classes mechanism in JavaScript in detail with an example. [5 Marks]

Ans:

Classes in JavaScript are a blueprint for creating objects. They were introduced in **ECMAScript 6 (ES6)** in 2015.

Key keywords:

- **class** — declares a class (name starts with capital letter)
- **constructor()** — special method called automatically when object is created

- **new** — creates an instance from a class
- **this** — refers to the current object

Syntax:

```
class ClassName {
  constructor(param1, param2) {
    this.param1 = param1;
  }
  methodName() { ... }
}
```

Example:

```
class Car {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }
  getDetails() {
    return this.name + ' - ' + this.year;
  }
}

const car1 = new Car('Ford', 2014);
console.log(car1.getDetails()); // Ford - 2014
```

Inheritance: Use **extends** and **super()** to inherit from another class.

```
class ElectricCar extends Car {
  constructor(name, year, range) {
    super(name, year);
    this.range = range;
  }
}
```

Q4. Write the Syntax and Program of any ten Array Methods in JavaScript. [5 Marks]

Ans:

JavaScript provides many built-in array methods. Here are 10 important ones:

Method	Description	Example	Output
push()	Add to end	arr.push(4)	[1,2,3,4]
pop()	Remove from end	arr.pop()	[1,2]
shift()	Remove from start	arr.shift()	[2,3]
unshift()	Add to start	arr.unshift(0)	[0,1,2,3]
length	Get array size	arr.length	3
concat()	Merge arrays	[1,2].concat([3,4])	[1,2,3,4]
slice(1,3)	Extract part	[10,20,30,40].slice(1,3)	[20,30]
forEach()	Loop through	arr.forEach(x=>log(x))	1 2 3
map()	Transform each	arr.map(x=>x*2)	[2,4,6]
filter()	Filter items	arr.filter(x=>x>2)	[3]

Q5. Write the Syntax and Program of any ten String Methods in JavaScript. [5 Marks]

Ans:

JavaScript strings are immutable — all methods return a new string.

Method	Description	Example	Output
length	String length	'Hello'.length	5
toUpperCase()	Convert to upper	'hello'.toUpperCase()	HELLO
toLowerCase()	Convert to lower	'HELLO'.toLowerCase()	hello
trim()	Remove spaces	' Hi '.trim()	Hi
charAt(1)	Char at index	'Hello'.charAt(1)	e
indexOf('lo')	Find position	'Hello'.indexOf('lo')	3
slice(0,4)	Extract part	'JavaScript'.slice(0,4)	Java
replace()	Replace text	'Hi JS'.replace('JS','World')	Hi World
includes()	Check if exists	'Hello World'.includes('World')	true
split(',')	Split to array	'a,b,c'.split(',')	['a','b','c']

Q6. Explain try block, catch block, and finally block in JavaScript with an example. [5 Marks]

Ans:

JavaScript's **try...catch...finally** is used to handle runtime errors without crashing the program.

try block: Wraps the code that might cause an error. JavaScript monitors this block for exceptions.

catch block: Executes only if an error is thrown inside try. It receives an error object with properties like **message**, **name**, and **stack**.

finally block: Always executes — whether an error occurred or not. Perfect for cleanup operations.

Complete Example:

```
function divide(a, b) {  
  try {  
    if (b === 0) throw new Error('Division by zero!');  
    console.log('Result: ' + a/b);  
  } catch(err) {  
    console.log('Error caught: ' + err.message);  
  } finally {  
    console.log('Operation complete.');  }  
}  
  
divide(10, 0);  
// Error caught: Division by zero!  
// Operation complete.
```

Error Handling in JavaScript is the process of detecting, managing, and responding to runtime errors so that the program does not crash unexpectedly.

Multiple catch is not directly supported, but you can check error type inside catch using **instanceof**.

MODULE – II

Asynchronous JavaScript — Callbacks, Promises, Timing

SHORT ANSWER QUESTIONS (2 Marks Each)

Q1. What problem does asynchronous JavaScript solve in single-threaded execution? [2 Marks]

Ans:

JavaScript is single-threaded — it executes one task at a time. Without async programming, long-running tasks (API calls, file reads) would **block** the entire program, freezing the UI.

Async JavaScript solves this by using an **Event Loop** that allows tasks to run in the background. Three main mechanisms: **Callbacks, Promises, and async/await**. The main thread stays free and handles the result only when the task finishes.

Q2. How does Synchronous programming work in JavaScript? [2 Marks]

Ans:

In **synchronous programming**, code executes line by line in order. Each statement must complete before the next one starts — it is a **blocking** model.

Example:

```
console.log('First');
console.log('Second');
console.log('Third');
// Output: First → Second → Third
```

Simple to read and debug, but if one task takes long (e.g., a slow API), the entire program waits and the UI freezes.

Q3. Explain the three Promise states in JavaScript (Pending, Fulfilled, Rejected). [2 Marks]

Ans:

A **Promise** is an object representing the eventual completion or failure of an asynchronous operation. It has 3 states:

- 1. Pending:** Initial state — operation is still running, result is undefined.
- 2. Fulfilled (Resolved):** Operation completed successfully — result value is available. Handled using **.then()**.
- 3. Rejected:** Operation failed — error reason is available. Handled using **.catch()**.

Once a promise settles (fulfilled/rejected), it cannot change state again.

Q4. Explain the methods that can resolve a Promise in JavaScript. [2 Marks]

Ans:

After creating a Promise, you handle its result using these methods:

.then(callback): Runs when the promise is **fulfilled** (resolve is called).

Syntax: `promise.then(result => console.log(result));`

.catch(callback): Runs when the promise is **rejected** (reject is called).

Syntax: `promise.catch(error => console.log(error));`

.finally(callback): Runs regardless of fulfilled or rejected — good for cleanup.

Syntax: `promise.finally(() => console.log('Done'));`

You can **chain** them: `promise.then(...).catch(...).finally(...);`

Q5. Explain briefly about states in Promise asynchronous operations. [2 Marks]

Ans:

Promises manage async operations through 3 states:

- **Pending** → operation is in progress
- **Fulfilled** → operation succeeded, `.then()` is triggered
- **Rejected** → operation failed, `.catch()` is triggered

Example:

```
const p = new Promise((resolve, reject) => {  
  let ok = true;  
  if(ok) resolve('Success!');  
  else reject('Failed!');  
});  
p.then(v => console.log(v)); // Success!
```

Q6. How does Asynchronous programming work in JavaScript? [2 Marks]

Ans:

Asynchronous programming allows the program to **start a task** and **continue executing** other code without waiting for that task to finish.

JavaScript achieves this through:

1. **Callbacks** — functions passed as arguments, called after task completes
2. **Promises** — objects representing future values
3. **async/await** — syntactic sugar over promises, makes async code look synchronous

The **Event Loop** monitors the call stack and callback queue, moving completed async tasks back to execution when the stack is empty.

LONG ANSWER QUESTIONS (5 Marks Each)

Q1. What is a Callback Function, and how does it enable asynchronous behaviour? Explain with an example. [5 Marks]

Ans:

A **callback function** is a function passed as an argument to another function, and is executed after the outer function completes its task.

Why Callbacks?

JavaScript is single-threaded. Callbacks allow code to execute **after** an async operation (like a timer or API call) finishes, without blocking the main thread.

Key Points:

- A function can receive another function as a parameter
- The callback runs after the outer operation finishes
- Enables non-blocking, async execution

Syntax:

```
function outerFunction(data, callback) {  
  // do task  
  callback(data); // call the callback  
}
```

Example:

```
function greet(name, callback) {  
  console.log('Hello ' + name);  
  callback();  
}  
  
function sayBye() {  
  console.log('Goodbye!');
```

```
}  
greet('Ravi', sayBye);  
// Output: Hello Ravi  
// Output: Goodbye!
```

Real-world async example with setTimeout:

```
console.log('Start');  
setTimeout(() => { console.log('After 2 seconds'); }, 2000);  
console.log('End');  
// Output: Start → End → After 2 seconds
```

Q2. Demonstrate how to implement time delays in JavaScript with setTimeout and setInterval. [5 Marks]

Timer functions are used to execute code after a specific delay or repeatedly at fixed time intervals. JavaScript provides two main timer functions:

1. setTimeout() — Executes a function **once** after a specified delay.

Syntax: let id = setTimeout(function, delayInMs);

Example:

```
function step1() { console.log('Step 1'); }  
function step2() { console.log('Step 2'); }  
setTimeout(step1, 3000); // runs after 3 seconds  
setTimeout(step2, 1000); // runs after 1 second  
// Output: Step 2 → Step 1
```

2. setInterval() — Executes a function **repeatedly** at fixed intervals.

Syntax: let id = setInterval(function, intervalInMs);

Example:

```
let id = setInterval(() => {  
  console.log('Executing every 1 second');  
}, 1000);
```

clearInterval() — Stops the setInterval.

```
setTimeout(() => { clearInterval(id); console.log('Stopped!'); }, 5000);
```

Key Differences:

- setTimeout → runs once
 - setInterval → runs repeatedly
 - 1 second = 1000 milliseconds
 - Always store the ID to stop execution later
-

Q3. How does overusing callbacks lead to unmaintainable code? Explain Callback Hell and solutions. [5 Marks]

Ans:

Callback Hell (Pyramid of Doom) occurs when multiple nested callbacks are used for sequential async operations, creating deeply indented code that is hard to read and maintain.

Example of Callback Hell:

```
setTimeout(() => {  
  console.log('Step 1');  
  setTimeout(() => {  
    console.log('Step 2');  
    setTimeout(() => {  
      console.log('Step 3');  
    }, 1000);  
  }, 1000);  
}, 1000);
```

```
}, 1000);  
}, 1000);
```

Problems:

- **Poor Readability:** Deep indentation, code grows horizontally
- **Difficult Maintenance:** Adding or changing logic is error-prone
- **Complex Error Handling:** Each nested level must handle errors separately
- **Hard to Debug:** Tracing execution flow is a nightmare

Solutions:

1. Named Functions — Break nested callbacks into separate named functions:

```
function step1() { console.log('Step 1'); setTimeout(step2, 1000); }  
function step2() { console.log('Step 2'); }  
setTimeout(step1, 1000);
```

2. Promises — Chain .then() calls instead of nesting:

```
doTask().then(() => doTask2()).then(() => doTask3()).catch(err => console.log(err));
```

3. async/await — Cleanest solution, reads like synchronous code:

```
async function runTasks() {  
  await doTask1();  
  await doTask2();  
  await doTask3();  
}
```

Q4. How do setTimeout and setInterval work? What are their differences? Explain with examples. [5 Marks]

Ans:

Both are JavaScript timing functions that schedule code execution asynchronously.

setTimeout(fn, delay) — Executes the function **once** after the given delay (in ms).

Example:

```
setTimeout(() => console.log('Runs once after 2s'), 2000);
```

setInterval(fn, interval) — Executes the function **repeatedly** at every interval (in ms) until stopped.

Example:

```
let id = setInterval(() => console.log('Repeating...'), 1000);  
// Stop after 5 seconds:  
setTimeout(() => clearInterval(id), 5000);
```

Feature	setTimeout	setInterval
Runs	Once	Repeatedly
Stop with	clearTimeout(id)	clearInterval(id)
Use case	Delayed one-time action	Clocks, polling
Triggered after	Delay from call	Fixed intervals

Q5. Explain the concept of 'Pyramid of Doom' in detail with an example program. [5 Marks]

Ans:

The **Pyramid of Doom** (also called Callback Hell) is a pattern where multiple nested callbacks form a deep pyramid/triangle-shaped indentation structure, making code extremely hard to read and maintain.

Occurs when: Multiple async operations depend on each other's results, so each new operation is nested inside the previous callback.

Example — 4 nested timeouts:

```
setTimeout(() => {  
  console.log('Step 1');  
  setTimeout(() => {  
    console.log('Step 2');  
    setTimeout(() => {  
      console.log('Step 3');  
      setTimeout(() => {  
        console.log('Step 4');  
      }, 1000);  
    }, 1000);  
  }, 1000);  
}, 1000);
```

Why it is a problem:

- Poor readability — code grows right instead of down
- Debugging is very difficult
- Error handling is repetitive at every level
- Very hard to maintain or modify

Best Solution — Use Named Functions:

```
function step1(){ console.log('Step 1'); setTimeout(step2,1000); }  
function step2(){ console.log('Step 2'); setTimeout(step3,1000); }  
function step3(){ console.log('Step 3'); }  
setTimeout(step1, 1000); // Clean and readable!
```

Even Better: Use Promises or async/await to eliminate callback hell completely.

Q6. How do Promises manage asynchronous operations? Explain their constructor structure with an example. [5 Marks]**Ans:**

A **Promise** is a JavaScript object that represents the eventual result of an asynchronous operation. It provides a cleaner alternative to callbacks.

Promise Constructor Syntax:

```
const myPromise = new Promise((resolve, reject) => {  
  // async task here  
  if (success) resolve(value); // fulfilled  
  else reject(error); // rejected  
});
```

Parameters:

- **resolve(value)** — Call this when task succeeds
- **reject(reason)** — Call this when task fails

Consuming a Promise:

- **.then(result => {})** — handles success
- **.catch(error => {})** — handles failure
- **.finally() => {}** — runs regardless

Complete Example:

```
const myPromise = new Promise((resolve, reject) => {  
  let success = true;  
  if(success) resolve('Task completed!');  
  else reject('Task failed!');  
});  
myPromise
```

```
.then(result => console.log(result))  
.catch(error => console.log(error));  
// Output: Task completed!
```

Promise Chaining solves callback hell:

```
fetch(url)  
.then(res => res.json())  
.then(data => console.log(data))  
.catch(err => console.log(err));
```

MODULE – III

React — JSX, Components, Props, Virtual DOM

SHORT ANSWER QUESTIONS (2 Marks Each)

Q1. How does JSX bridge the gap between HTML markup and JavaScript logic in React? [2 Marks]

Ans:

JSX (JavaScript XML) is a syntax extension that allows writing **HTML-like code directly inside JavaScript**. It bridges the gap by letting developers define UI structure and logic in the same file.

How it works:

- JSX looks like HTML but is actually JavaScript
- A transpiler (Babel) converts JSX into `React.createElement()` calls
- JavaScript expressions can be embedded using curly braces `{ }`

Example: `const el = <h1>Hello {name}!</h1>;`

Q2. Explain the core concepts of React and its significance in modern front-end development. [2 Marks]

Ans:

React is a JavaScript **UI library** developed by Meta (Facebook) for building fast, interactive web apps. Core concepts:

1. **JSX** — write HTML-like syntax inside JavaScript
2. **Virtual DOM** — lightweight copy of real DOM, enables fast updates
3. **Components** — reusable, self-contained UI pieces
4. **Props** — pass data from parent to child
5. **State** — internal data that re-renders the component when changed
6. **Unidirectional Data Flow** — data flows parent → child, makes debugging easier

Q3. What role does JSX play in React development, and how does it simplify component creation? [2 Marks]

Ans:

JSX simplifies component creation by allowing HTML structure and JavaScript logic to coexist in the same file, making UI code more **readable and intuitive**.

Key JSX rules:

- Must return one root element (use `<>...</>` fragments)
- All tags must be closed (`<img />`, `<input />`)
- Use **className** instead of `class`
- Embed JS expressions using `{ }`

Example: `const name='Alice'; return <h1>Hello {name}</h1>;`

Q4. Explain the role of the Virtual DOM in updating user interfaces efficiently. [2 Marks]

Ans:

The **Virtual DOM** is a lightweight, in-memory representation of the real DOM. React uses it to make UI updates fast.

Process:

1. State/Props change → New Virtual DOM is created
2. React **diffs** (compares) new vs old Virtual DOM
3. Only the changed parts are updated in the **Real DOM**

This avoids unnecessary re-rendering of unchanged elements, making React much faster than direct DOM manipulation.

Q5. What are the differences between functional components and class components in React? [2 Marks]

Ans:

Key differences:

Feature	Functional Component	Class Component
Syntax	Simple function	ES6 class
State	useState Hook	this.state
Lifecycle	useEffect Hook	Lifecycle methods
this keyword	Not used	Required
Performance	Faster, lighter	Slightly heavier
Modern Use	Preferred (ES6+)	Legacy / older code

Q6. Explain the role of props in React and why a child component cannot change its own props. [2 Marks]

Ans:

Props (Properties) are used to pass data from a **parent component** to a **child component**. They make components dynamic and reusable.

Why props are read-only (immutable)?

React enforces unidirectional data flow — data flows **parent** → **child** only. If a child could modify its props, it would break the predictable data flow, making debugging extremely difficult.

Example:

```
function Welcome(props) { return <h1>Hello {props.name}</h1>; }  
<Welcome name='Alice' /> // Output: Hello Alice
```

LONG ANSWER QUESTIONS (5 Marks Each)

Q1. What are the core principles and features of the React library? Explain with an example. [5 Marks]

Ans:

React is a JavaScript library for building **user interfaces**. It focuses only on the view layer and uses a component-based architecture.

Core Principles:

1. JSX: Write HTML-like syntax inside JavaScript.

```
const el = <h1>Hello World</h1>;
```

2. Virtual DOM: React updates only the changed parts of UI for performance.

3. Components: Reusable, independent UI pieces (functional or class).

4. Props: Pass data from parent to child. Read-only.

5. State: Internal data that triggers re-renders when changed.

```
const [count, setCount] = useState(0);
```

6. Unidirectional Data Flow: Data flows parent → child only, making apps predictable.

7. Hooks: useState, useEffect, etc. allow functional components to use state and lifecycle features.

Example (Props):

```
function Student(props) {  
  return <h1>Name: {props.name}, Roll: {props.roll}</h1>;  
}  
// In App.js:  
<Student name='Ravi' roll={101} />
```

// Output: Name: Ravi, Roll: 101

Q2. What are the different ways to create components in React and when should you use them? [5 Marks]

Ans:

React has two main types of components:

1. Functional Components (Modern, Preferred):

A simple JavaScript function that returns JSX.

Basic Syntax:

```
function Welcome() {  
  return <h1>Hello Students!</h1>;  
}  
export default Welcome;
```

Arrow Function syntax:

```
const Welcome = () => <h1>Hello Students!</h1>;
```

With Props:

```
function Card(props) {  
  return <div><h2>{props.name}</h2></div>;  
}  
<Card name='Alice' />
```

With Hooks (useState):

```
import { useState } from 'react';  
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count+1)}>Count: {count}</button>;  
}
```

2. Class Components (Legacy):

Created using ES6 class. Must extend Component and have a render() method.

Syntax:

```
import React, { Component } from 'react';  
class Welcome extends Component {  
  render() { return <h1>Hello!</h1>; }  
}
```

When to use: Use **Functional Components** for all new code. Use Class Components only when working with older codebases.

Q3. Compare functional components and class components in React with examples. [5 Marks]

Ans:

Both types create UI components, but differ significantly in syntax and features:

Aspect	Functional Component	Class Component
Definition	JavaScript function	ES6 class extending Component
Render	Returns JSX directly	Uses render() method
State	useState hook	this.state & this.setState()
Lifecycle	useEffect hook	componentDidMount etc.
this keyword	Not needed	Required for props/state
Code size	Shorter, cleaner	More boilerplate

Ans:

Functional Component Example:

```
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count+1)}>{count}</button>;
}
```

Class Component Equivalent:

```
class Counter extends Component {
  constructor() { super(); this.state = { count: 0 }; }
  render() {
    return <button onClick={() => this.setState({count: this.state.count+1})}>{this.state.count}</button>;
  }
}
```

Conclusion: Functional Components with Hooks are the modern standard in React.

Q4. Explain the Component Lifecycle in detail, covering Mounting, Updating, and Unmounting phases. [5 Marks]

Ans:

React class components go through a lifecycle from creation to removal. It has 3 phases:

1. MOUNTING PHASE — Component is created and inserted into the DOM.

Methods (in order):

- constructor()** — Initializes state and binds methods. Runs first.
- render()** — Returns JSX to display.
- componentDidMount()** — Runs after component appears on screen. Used for API calls, timers.

2. UPDATING PHASE — Component re-renders when state/props change.

Methods (in order):

- shouldComponentUpdate()** — Returns true/false to allow/prevent re-render.
- render()** — Re-renders the updated UI.
- componentDidUpdate()** — Runs after update. Good for post-update side effects.

3. UNMOUNTING PHASE — Component is removed from the DOM.

Method:

- componentWillUnmount()** — Runs just before component is destroyed. Used to stop timers, cancel API calls, remove event listeners.

In Functional Components: All lifecycle phases are handled by the **useEffect** hook:

```
useEffect(() => {
  // componentDidMount code
  return () => { /* componentWillUnmount cleanup */ };
}, []); // [] = run once on mount
```

Q5. What is JSX? Write a code snippet demonstrating how to embed JavaScript expressions and attributes within JSX. [5 Marks]

Ans:

JSX (JavaScript XML) is a syntax extension for JavaScript that lets you write HTML-like markup inside JavaScript. It is the standard way to define UI in React.

Core Concepts:

- JSX is compiled by **Babel** into `React.createElement()` calls
- JSX expressions evaluate to **React Elements**
- Enables component-based UI development

JSX Syntax Rules:

1. Return one root element — use `<>...</>` fragment if needed

2. Close all tags: `<img />`, `<br />`
3. Use **className** instead of class
4. Embed JS using curly braces `{ }`
5. Component names must start with capital letter

Embedding Expressions:

```
function Profile() {
  const name = 'Ravi';
  const age = 21;
  const isStudent = true;
  return (
    <>
    <h1>Hello, {name}!</h1>
    <p>Age: {age * 1}</p>
    <p>{isStudent ? 'Student' : 'Not a student'}</p>
    <img src='photo.jpg' className='profile-pic' />
    </>
  );
}
```

Attributes in JSX:

- HTML: class → JSX: className
- HTML: for → JSX: htmlFor
- HTML: onclick → JSX: onClick
- Dynamic attribute: `<input value={username} onChange={handleChange} />`

Q6. Describe the process of Props Communication between a parent and a child component with a code snippet. [5 Marks]

Ans:

Props Communication allows data to flow from parent to child components. React follows **Unidirectional Data Flow** — data moves only parent → child.

Key Points:

- Props are read-only (child cannot modify them)
- Passed as HTML attributes on the component tag
- Accessed in child via `props.propName`

Parent to Child Example:

```
// Child.jsx
function Student(props) {
  return (
    <div>
    <h2>Name: {props.name}</h2>
    <p>Roll: {props.roll}</p>
    </div>
  );
}

// App.jsx (Parent)
function App() {
  return <Student name='Alice' roll={42} />;
}

// Output: Name: Alice, Roll: 42
```

Child to Parent Communication (Callback Props):

When a child needs to send data back, the parent passes a **function as a prop**:

Parent.jsx:

```
function Parent() {  
  function receive(msg) { alert(msg); }  
  return <Child send={receive} />;  
}
```

Child.jsx:

```
function Child(props) {  
  return <button onClick={() => props.send('Hello Parent!')}>Send</button>;  
}
```

Complete Flow: App.jsx → Parent.jsx → Child.jsx (props flow down, callbacks flow up)

MODULE – IV

React Hooks — useState, useEffect, useReducer, Forms, Events

SHORT ANSWER QUESTIONS (2 Marks Each)

Q1. How do you add state management capabilities to a functional React component? [2 Marks]

Ans:

In functional components, state is managed using **React Hooks**:

1. useState — for simple state (strings, numbers, booleans):

```
const [count, setCount] = useState(0);
```

2. useReducer — for complex state with multiple values:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

3. useContext — for global state shared across many components:

```
const data = useContext(MyContext);
```

Call **setCount(newValue)** to update state. React automatically re-renders the component.

Q2. What constitutes a side effect, and how can you handle it in functional components? [2 Marks]

Ans:

A **side effect** is any operation that interacts with something outside the component — not just rendering the UI.

Common side effects: API/data fetching, timers, DOM manipulation, event subscriptions, logging.

Handle side effects using the **useEffect** hook:

```
useEffect(() => {  
  // side effect code here  
  return () => { /* cleanup */ };  
}, [dependencies]);
```

The dependency array controls when the effect runs: empty [] = once on mount, [value] = whenever value changes.

Q3. What is the role of keys in React's reconciliation (diffing) algorithm? [2 Marks]

Ans:

Whenever new elements are added to the UI, a virtual DOM is created.

A Virtual DOM is a JS object, which is the virtual representation of an HTML DOM.

React compares new virtual DOM with current virtual DOM, and the difference will be efficiently updated to HTML DOM. So, the Virtual DOM helps to render the UI more performantly.

React only updates what's necessary. This process is known as **Reconciliation**.

Q4. Explain the performance and functional tradeoffs between controlled and uncontrolled inputs in React. [2 Marks]

Ans:

Two patterns for handling form inputs in React:

If the Input Element value is handled by a React State, then it is called

Controlled Input

If the Input Element value is handled by the browser itself, then it is called

Uncontrolled Input

It is Suggested that react state is better way to handle the Input Element value.

Access value	From state	Via ref.current.value
Use case	Complex forms, validation	Simple, fast forms

Q5. What are the cleanup steps for useEffect Hook to prevent memory leaks? [2 Marks]

Ans:

In React, **cleanup steps** are used to **stop or remove side effects** when a component is unmounted or before the effect runs again. This helps to **prevent memory leaks**.

Why to do cleanup :

- 1) To stop **timers (setInterval, setTimeout)**
- 2) To remove **event listeners**
- 3) To cancel **API requests**
- 4) To avoid unnecessary background processes

How to do a clean up :

Inside useEffect, return a **cleanup function**

Common cleanups: clearInterval(), clearTimeout(), removeEventListener(), cancel API requests

LONG ANSWER QUESTIONS (5 Marks Each)

Q1. Explain the useReducer hook and how it differs from useState for complex state logic. [5 Marks]

Ans:

The **useReducer** hook is an alternative to useState for managing **complex state logic**. It follows the Redux pattern — state is updated by dispatching actions to a reducer function.

Syntax:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Key Parts:

- **state** — current state object
- **dispatch** — function to send action objects
- **reducer** — pure function: (state, action) => newState
- **initialState** — starting value of state

Example — Counter with useReducer:

```
function counterReducer(state, action) {
  switch(action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    case 'reset': return { count: 0 };
    default: throw new Error('Unknown action');
  }
}

function Counter() {
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });
  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({type:'increment'})}>+</button>
      <button onClick={() => dispatch({type:'decrement'})}>-</button>
      <button onClick={() => dispatch({type:'reset'})}>Reset</button>
    </div>
  );
}
```

}

Feature	useState	useReducer
Best for	Simple state	Complex state logic
Update method	setState(newValue)	dispatch({type:'action'})
Logic location	Inside component	Separate reducer function
Scalability	Harder with many states	Easier with many states

Q2. Demonstrate how to create and implement a Custom Hook to reuse logic across components. [5 Marks]

Ans:

A **Custom Hook** is a regular JavaScript function whose name starts with 'use'. It allows you to extract and reuse stateful logic across multiple components.

Why Custom Hooks?

- Avoid repeating the same useState/useEffect logic in multiple components
- Keep components clean and focused on UI only
- Easily testable and sharable logic

Step 1 — Create the Custom Hook (useCounter.js):

```
import { useState } from 'react';
function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);
  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);
  const reset = () => setCount(initialValue);
  return { count, increment, decrement, reset };
}
export default useCounter;
```

Step 2 — Use it in Component A:

```
import useCounter from './useCounter';
function ComponentA() {
  const { count, increment, reset } = useCounter(10);
  return (
    <div>
      <p>Counter A: {count}</p>
      <button onClick={increment}>+</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}
```

Step 3 — Reuse in Component B (separate state instance):

```
function ComponentB() {
  const { count, increment } = useCounter(0);
  return <button onClick={increment}>B: {count}</button>;
}
```

Key point: Each component gets its own independent state — custom hooks don't share state between components.

Q3. Describe the useEffect Hook in detail — how does the dependency array control the execution of side effects? [5 Marks]

Ans:

The **useEffect** hook lets you perform side effects in functional components. It runs after the component renders.

Syntax:

```
useEffect(() => {  
  // side effect code  
  return () => { /* cleanup */ };  
}, [dependency1, dependency2]);
```

Dependency Array — 3 Modes:

Mode 1 — No dependency array: Runs after **every render**.

```
useEffect(() => { console.log('Runs every render'); });
```

Mode 2 — Empty array []: Runs **once** after initial mount only.

```
useEffect(() => {  
  fetch('/api/data').then(r => r.json()).then(setData);  
}, []); // Like componentDidMount
```

Mode 3 — With values [dep]: Runs on mount AND whenever dep changes.

```
useEffect(() => {  
  console.log('userId changed:', userId);  
}, [userId]); // Refetch when userId changes
```

Cleanup function: Return a function to clean up (stop timers, remove listeners):

```
useEffect(() => {  
  const timer = setInterval(() => setCount(c => c+1), 1000);  
  return () => clearInterval(timer); // runs on unmount  
}, []);
```

Common Use Cases:

- [] → Fetch initial data
- [id] → Refetch when ID changes
- No array → Sync with every state change

Q4. Discuss Conditional Rendering techniques in React (Ternary operators, Logical &&). [5 Marks]**Ans:**

Conditional rendering lets you display different UI based on conditions. React uses standard JavaScript techniques:

1. if/else Statement — Used outside JSX for complex conditions:

```
function Greeting({ isLoggedIn }) {  
  if (isLoggedIn) return <h1>Welcome back!</h1>;  
  else return <h1>Please sign up.</h1>;  
}
```

2. Ternary Operator (? :) — Best for simple if/else inside JSX:

```
return (  
  <div>  
    {isLoggedIn ? <h1>Welcome!</h1> : <h1>Please login.</h1>}  
  </div>  
)
```

3. Logical AND (&&) — Renders something only when condition is true:

```
return (  
  <div>  
    {hasMessages && <p>You have new messages!</p>}  
  </div>  
)
```

4. Switch Statement — For multiple conditions/cases:

```
function renderStatus(status) {  
  switch(status) {  
    case 'loading': return <p>Loading...</p>;  
    case 'success': return <h1>Data Loaded!</h1>;  
    case 'error': return <p>Error!</p>;  
    default: return null;  
  }  
}
```

Summary: Use **&&** for show/hide, **?** : for if/else, **if/else** for complex logic outside JSX.

Q5. Discuss Event Handling Techniques in React. How do you pass arguments to an event handler without triggering it on render? [5 Marks]

Ans:

React uses **camelCase** event names and passes functions (not strings) as handlers.

Basic Event Handling:

```
function handleClick() { alert('Clicked!'); }  
<button onClick={handleClick}>Click Me</button>
```

Common Events: `onClick`, `onChange`, `onSubmit`, `onMouseOver`, `onKeyDown`

Passing Arguments — The Problem:

■ Wrong: `<button onClick={handleClick(id)}>` // triggers on render!

■ Right: wrap in arrow function

Solution 1 — Arrow Function (Most Common):

```
<button onClick={() => handleClick(id)}>Click</button>
```

The arrow function is a new function that calls `handleClick` only when clicked.

Solution 2 — `bind()` Method:

```
<button onClick={handleClick.bind(null, id)}>Click</button>
```

Solution 3 — Currying (Higher Order Function):

```
const handleClick = (id) => (event) => { console.log(id, event); };  
<button onClick={handleClick(id)}>Click</button>
```

Accessing the event object:

```
function handleChange(e) { console.log(e.target.value); }  
<input onChange={handleChange} />
```

Q6. What are the best practices for handling form submission and input data in React? [5 Marks]

Ans:

React has two patterns for managing form input data:

1. Controlled Components (Standard Approach):

React state is the single source of truth. Every keystroke updates state.

Example:

```
function LoginForm() {  
  const [formData, setFormData] = useState({ username: "", email: "" });  
  const handleChange = (e) => {  
    const { name, value } = e.target;  
    setFormData(prev => ({ ...prev, [name]: value }));  
  };  
  const handleSubmit = (e) => {  
    e.preventDefault(); // prevent page reload  
    console.log(formData);  
  };  
}
```

```

};
return (
  <form onSubmit={handleSubmit}>
    <input name='username' value={formData.username} onChange={handleChange} />
    <input name='email' value={formData.email} onChange={handleChange} />
    <button type='submit'>Submit</button>
  </form>
);
}

```

2. Uncontrolled Components (Ref-Driven):

DOM manages data. Pull value only on submit using refs. Faster, fewer re-renders.

Example:

```

function UncontrolledForm() {
  const inputRef = useRef();
  return (
    <form onSubmit={(e) => { e.preventDefault(); console.log(inputRef.current.value); }}>
      <input type='text' ref={inputRef} />
      <button type='submit'>Submit</button>
    </form>
  );
}

```

Best Practices:

- Always use `e.preventDefault()` to stop page reload
- Use controlled components for validation
- Use a single `handleChange` for all fields using `e.target.name`
- Clear form after submit: `setFormData({ username: "", email: "" })`

MODULE – V

React Router, Context API, State Management, Authentication

SHORT ANSWER QUESTIONS (2 Marks Each)

Q1. What is the role of BrowserRouter and Route in React Router? [2 Marks]

Ans:

BrowserRouter: A wrapper component that uses the HTML5 History API to keep your UI in sync with the URL. It must wrap the entire application to enable routing.

Route: Defines a mapping between a URL path and a React component. When the current URL matches the path, the component is rendered.

Example:

```
function App() {  
  return (  
    <BrowserRouter>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

Q2. How do you extract values using the useParams() hook? [2 Marks]

Ans:

useParams() is a React Router hook used in functional components to access dynamic URL parameters (defined as :paramName in the Route path).

Syntax: const { paramName } = useParams();

Example:

```
// Route definition:  
<Route path="/user/:id" element={<User />} />  
// Inside User component:  
function User() {  
  const { id } = useParams();  
  return <h2>User ID: {id}</h2>;  
}  
// URL /user/42 → Output: User ID: 42
```

Q3. Define the Context API and its role in global state management. [2 Marks]

Ans:

The **Context API** is a built-in React feature that allows sharing data (state) across the entire component tree without passing props through every level.

Role in Global State: Provides a central 'store' (Provider) that any nested component can access directly using useContext(), eliminating prop drilling.

3 Steps:

1. **Create:** const MyCtx = createContext();
2. **Provide:** <MyCtx.Provider value={data}>...</MyCtx.Provider>
3. **Consume:** const data = useContext(MyCtx);

Q4. What is Client-Server Communication in the context of a React app? [2 Marks]

Ans:

Client-Server Communication refers to the process where a React application (client) sends requests to a backend API/server to **fetch, create, update, or delete data**.

Mechanism: Handled using the built-in **fetch()** API or the **Axios** library, typically inside a **useEffect** hook.

Example using fetch:

```
useEffect(() => {  
  fetch('https://api.example.com/users')  
    .then(res => res.json())  
    .then(data => setUsers(data));  
}, []);
```

Q5. What are Nested Routes, and when should they be used? [2 Marks]

Ans:

Nested Routes are routes defined inside a parent route. They allow a portion of the UI to change while the parent layout stays constant.

Syntax:

```
<Route path='/dashboard' element={<Dashboard />}>  
  <Route path='profile' element={<Profile />} />  
  <Route path='settings' element={<Settings />} />  
</Route>
```

When to use: When sub-views depend on a parent view (e.g., Dashboard with Profile and Settings tabs). The parent component uses **<Outlet />** to render the active child route.

LONG ANSWER QUESTIONS (5 Marks Each)

Q1. Compare the Context API with the Reducer Pattern for managing advanced application state. [5 Marks]

Ans:

Both are state management solutions. Here's a detailed comparison:

Feature	Context API	Reducer Pattern (useReducer)
Purpose	Share global data; avoid prop drilling	Handle complex state logic with actions
Works with	useContext hook	useReducer hook
State update	Direct setState calls	Dispatch actions → reducer → new state
Complexity	Simple and easy	Slightly more code (actions, reducer)
Best for	Themes, auth, language settings	Counters, forms, multi-step workflows
Predictability	Moderate	High (reducer is a pure function)
Combined use	Context distributes state	useReducer manages the state logic

Ans:

Best Practice: Use them **together** — useReducer manages complex state, and Context API distributes it globally:

```
const [state, dispatch] = useReducer(reducer, initialState);  
return (  
  <AppContext.Provider value={{ state, dispatch }}>  
    {children}  
  </AppContext.Provider>  
);
```

Conclusion: Context API solves **where** state is shared. Reducer Pattern solves **how** state changes.

Q2. Explain the implementation of Application Routing, including navigation components like Link and NavLink. [5 Marks]

Ans:

Application Routing in React allows navigation between different views/pages without full page reload. Implemented using the **react-router-dom** library.

Installation: npm install react-router-dom

Step 1 — Setup BrowserRouter and Routes:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
function App() {
  return (
    <BrowserRouter>
    <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="/contact" element={<Contact />} />
    </Routes>
    </BrowserRouter>
  );
}
```

Step 2 — Link Component: Navigates between routes without reloading the page (like an anchor tag).

```
import { Link } from 'react-router-dom';
function Navbar() {
  return (
    <nav>
    <Link to="/">Home</Link>
    <Link to="/about">About</Link>
    </nav>
  );
}
```

Step 3 — NavLink Component: Like Link but automatically adds an 'active' class to the currently active route link.

```
import { NavLink } from 'react-router-dom';
function Navbar() {
  return (
    <nav>
    <NavLink to="/" className={({ isActive }) => isActive ? 'active' : ""}>Home</NavLink>
    <NavLink to="/about" className={({ isActive }) => isActive ? 'active' : ""}>About</NavLink>
    </nav>
  );
}
```

Key Differences: Link vs NavLink

- Both prevent page reload
- NavLink provides isActive state for styling the active link
- Use NavLink for navigation menus/tabs; use Link for general links

Q3. Describe the general workflow for implementing User Authentication in a React application. [5 Marks]

Ans:

User authentication ensures only authorized users access protected parts of the app. Here is the complete workflow:

Step 1 — User Input (Login/Signup Form):

Capture credentials (email, password) using controlled components, then send via POST request to backend.

Step 2 — Backend Verification:

Server checks credentials → if valid, generates a **JWT (JSON Web Token)** → sends token back to client.

Step 3 — Client-Side Token Storage:

Store the token in **localStorage** or **sessionStorage**:

```
localStorage.setItem('token', receivedToken);
```

Step 4 — Global Auth State (Context API):

Use Context API to manage a global **isAuthenticated** state and user info accessible anywhere in the app.

Step 5 — Protected Routes:

Create a ProtectedRoute component that redirects unauthenticated users to login:

```
function ProtectedRoute({ element }) {  
  const token = localStorage.getItem('token');  
  return token ? element : <Navigate to="/login" />;  
}  
  
<Route path="/dashboard" element={<ProtectedRoute element={<Dashboard />} />} />
```

Step 6 — Authenticated API Requests:

Attach token to request headers:

```
fetch('/api/data', { headers: { Authorization: 'Bearer ' + token } });
```

Step 7 — Logout:

Clear storage and reset auth state:

```
localStorage.removeItem('token');  
setIsAuthenticated(false);  
navigate('/login');
```

Q4. Discuss Advanced State Management using the Context API to solve the 'Prop Drilling' problem. [5 Marks]

Ans:

Prop Drilling occurs when data is passed through multiple layers of components just to reach a deeply nested component, even if intermediate components don't use that data.

Problem Example:

App → Parent → Child → GrandChild (needs theme)

Without Context: theme must be passed through Parent and Child even if they don't use it.

Why is it a problem?

- Code becomes cluttered with unused props
- Hard to maintain — changing prop name requires changes everywhere
- Components become tightly coupled

Solution — Context API:

Step 1: Create Context

```
import { createContext } from 'react';  
const ThemeContext = createContext();
```

Step 2: Provide Context (App.jsx)

```
function App() {  
  const [theme, setTheme] = useState('light');  
  return (  
    <ThemeContext.Provider value={{ theme, setTheme }}>  
    <MainContent />  
  );  
}
```

```

</ThemeContext.Provider>
);
}

```

Step 3: Consume Context (GrandChild.jsx — directly)

```

import { useContext } from 'react';
function GrandChild() {
  const { theme } = useContext(ThemeContext);
  return <div>Current Theme: {theme}</div>;
}

```

Benefits:

- No prop drilling — GrandChild accesses theme directly
- Cleaner, maintainable code
- Built-in React feature (no extra library needed)

Q5. Explain the Route Setup and Implementation process in React Router, including BrowserRouter, Routes, and Route. [5 Marks]

Ans:

React Router enables navigation between views in a Single Page Application (SPA) without page reloads.

Install with: **npm install react-router-dom**

1. BrowserRouter:

Wraps the entire app. Uses HTML5 History API to sync URL with UI. Must be at the top level.

2. Routes:

A container for all Route components. Checks the current URL and renders the matching Route.

3. Route:

Maps a URL path to a React component via the **path** and **element** props.

Complete Setup Example:

```

import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import Home from './Home';
import About from './About';
import NotFound from './NotFound';
function App() {
  return (
    <BrowserRouter>
    <nav>
    <Link to="/">Home</Link> | <Link to="/about">About</Link>
    </nav>
    <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="*" element={<NotFound />} />
    </Routes>
    </BrowserRouter>
  );
}

```

Dynamic Routes with useParams:

```

<Route path="/user/:id" element={<UserProfile />} />
// In UserProfile: const { id } = useParams();

```

Nested Routes with Outlet:

```

<Route path="/dashboard" element={<Dashboard />}>

```

```

<Route path='stats' element={<Stats />} />
</Route>
// Dashboard must render <Outlet /> where child routes appear

```

Q6. Explain the workflow of Client-Server Communication using fetch or Axios within a component's lifecycle. [5 Marks]

Ans:

Client-Server Communication allows React apps to interact with backend APIs. The two main tools are the built-in **fetch()** API and the **Axios** library.

Using fetch() inside useEffect:

```

import { useState, useEffect } from 'react';
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(res => {
        if(!res.ok) throw new Error('Network error');
        return res.json();
      })
      .then(data => { setUsers(data); setLoading(false); })
      .catch(err => { setError(err.message); setLoading(false); });
    }, []); // runs once on mount
    if(loading) return <p>Loading...</p>;
    if(error) return <p>Error: {error}</p>;
    return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
  }

```

Using Axios (simpler syntax):

```

import axios from 'axios';
useEffect(() => {
  axios.get('https://api.example.com/users')
    .then(res => setUsers(res.data))
    .catch(err => setError(err.message));
}, []);

```

POST Request (Create data):

```

const handleSubmit = async () => {
  const res = await fetch('/api/users', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name: 'Alice' })
  });
  const data = await res.json();
  console.log(data);
};

```

Key Points:

- Always use useEffect for initial data fetching
- Show loading/error states to improve UX
- Use async/await or .then() chains
- fetch vs Axios: Axios auto-parses JSON, has better error handling

END OF QUESTION BANK

All the Best for Your Exams!

Malla Reddy Technical Campus — WAD-II